Natural Language Processing

Tokenization: French vs Japanese

How splitting text into words works when a language does not use spaces

French	`je` `suis` `un` `chat`
Japanese, split on spaces	`吾輩は猫である`
Japanese, MeCab	`吾輩` `は` `猫` `で` `ある`

Joah Emy-Nakane

October 2026

1 Introduction

Before a model can read text, the text has to be cut into *tokens*, and every token gets an integer id from a vocabulary. In French, the obvious way is to cut on spaces. Japanese does not put spaces between words, so that rule does not work.

This report compares the two languages in three steps:

1. a toy corpus of five sentences, passed through `keras.layers.TextVectorization`, the layer used in the course notebook;
2. a real corpus of 38,292 French–Japanese sentence pairs from Tatoeba, with the Japanese side split by the morphological analyzer MeCab;
3. the same pairs passed through two multilingual subword tokenizers.

All the code is in the repository: `french/`, `japanese/` and `corpus/`.

2 A toy example

Both corpora contain five short sentences. The vocabulary is built from the corpus with `adapt()`, then a test sentence that is not in the corpus is encoded and decoded.

2.1 French

By default, `TextVectorization` lowercases the text, strips punctuation and splits on whitespace. In French, that directly gives words.

Vocabulary	`[UNK]` `je` `un` `suis` `texte` `écrivain` `écrire` `si` `sait` `pense` `pas` `ne` `chat` `autre`
Sentence	Je suis un chat qui sait écrire
Encoded	`[2, 4, 3, 13, 1, 9, 7]`
Decoded	`je` `suis` `un` `chat` `[UNK]` `sait` `écrire`

Only *qui*, which is not in the corpus, becomes `[UNK]`. (Index 0 is kept for padding and index 1 for `[UNK]`.)

2.2 Japanese, split on spaces

With the same settings, each Japanese sentence is a single token, because it contains no space:

Vocabulary	`[UNK]` `私は猫ではありません` `私は別のテキストです` `私はテキストです` `我思う、ゆえに我あり` `作家は書くことができる`
Sentence	私は書くことができる猫です
Encoded	`[1]`
Decoded	`[UNK]`

The test sentence never appears as-is in the corpus, so the whole sentence is unknown, even though it reuses known pieces: 私は (*I*), 書くことができる (*can write*) and 猫 (*cat*).

2.3 Japanese, split by character

With `split="character"`, every character is a token:

Vocabulary `[UNK]` は で 私 我 ト テ ス キ り す あ 猫 書 思 家 別 作 ん る ゆ ま の に と せ こ く き が え う 、

Encoded `[4, 2, 14, 28, 27, 25, 30, 3, 29, 20, 13, 3, 11]`

Decoded 私 は 書 く こ と が で き る 猫 で す

There is no `[UNK]` any more, but the notion of a word is gone: テキスト (*text*) is split into four characters. Note also that 、 stays in the vocabulary: the default standardization only strips ASCII punctuation.

2.4 Japanese with MeCab

MeCab is a morphological analyzer. It uses a dictionary (here UniDic, installed with `unidic-lite`) to cut the text into words and tag each one with its part of speech:

Token	Tag	
私	代名詞	pronoun
は	助詞	particle
書く	動詞	verb
こと	名詞	noun
が	助詞	particle
できる	動詞	verb
猫	名詞	noun
です	助動詞	auxiliary verb

With the `-Owakati` option, MeCab only outputs the words separated by spaces. That text can then go through `TextVectorization` with the default settings, exactly like French:

Vocabulary `[UNK]` は 私 我 テキスト です あり 猫 書く 思う 別 作家 ん ゆえ ませ の に できる で こと が 、

Sentence 私は書くことができる猫です

Encoded `[3, 2, 9, 20, 21, 18, 8, 6]`

Decoded 私 は 書く こと が できる 猫 です

No `[UNK]`, and the tokens are words again: テキスト stays in one piece, 書く (*to write*) and 猫 (*cat*) are recognized.

2.5 Summary

Method on 私は書くことができる猫です	Tokens	`[UNK]`	Tokens are
Whitespace split	1	1	whole sentences
Character split	13	0	single characters
MeCab, then whitespace split	8	0	words

Five sentences are enough to see that Japanese needs an extra step before the usual pipeline. They are not enough to say how the result compares to French on real text.

3 A real corpus: Tatoeba

3.1 Setup

Tatoeba (`https://tatoeba.org`) is a collection of sentences translated by volunteers. Its exports give 43,856 links between a French and a Japanese sentence. When a sentence has several translations, only one pair is kept, so both sides have the same number of distinct sentences: **38,292 pairs**, shuffled and split into 30,633 for training and 7,659 for testing. The sentences are short and everyday:

French Nous nous sommes dépêchés pour attraper le bus.

Japanese 私たちはバスに乗るために急いだ。

Because the content is the same on both sides, the differences come from the languages and from how they are split, not from the topics.

- French is split on whitespace, as `TextVectorization` does by default.
- Japanese is first split by MeCab (`-Owakati`), then on whitespace.
- In both languages, the text is lowercased and *all* punctuation and symbols are removed (regex `[\p{P}\p{S}]`), so that 。 and 、 are not counted as Japanese words.

The script is `corpus/compare.py`.

3.2 Sentence length

Mean per sentence	Characters	Tokens	Characters per token
French	33.7	7.6	4.45
Japanese	15.6	9.8	1.60

Japanese says the same thing with half as many characters, because a kanji carries a lot of meaning (急 alone means *hurry*). But it needs *more* tokens: MeCab cuts out every particle and every auxiliary, which French either does not have or glues to the word.

3.3 Vocabulary growth

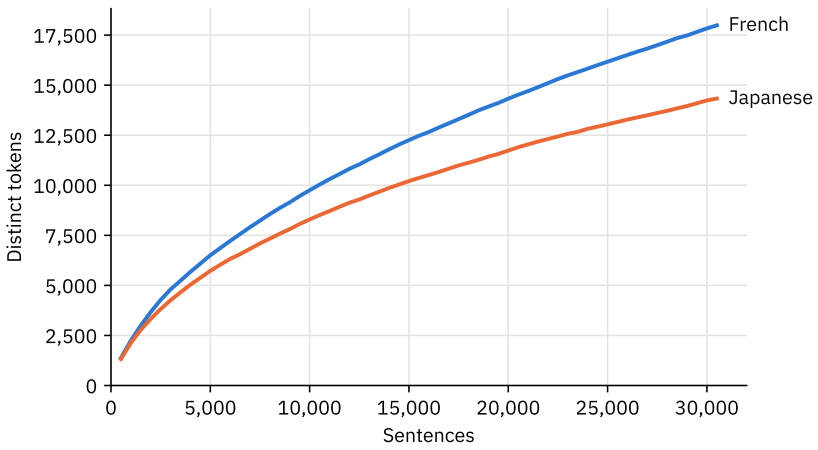


Figure 1: Number of distinct tokens as sentences are added (training set).

On the 30,633 training sentences, French ends with **18,025** distinct tokens and Japanese with **14,362**. Half of them are seen only once (52 % in French, 47 % in Japanese). Both curves keep rising, but the French one rises faster: every verb has many surface forms (*écris*, *écrit*, *écrivons*, *écrivait*⋯), each one a different token. In Japanese, MeCab separates part of the conjugation into its own tokens (急い + だ), so the same pieces come back more often.

3.4 Unknown tokens

A model has a fixed vocabulary (`max_tokens`). The vocabulary is built on the training set, then we count how many tokens of the test set fall outside of it.

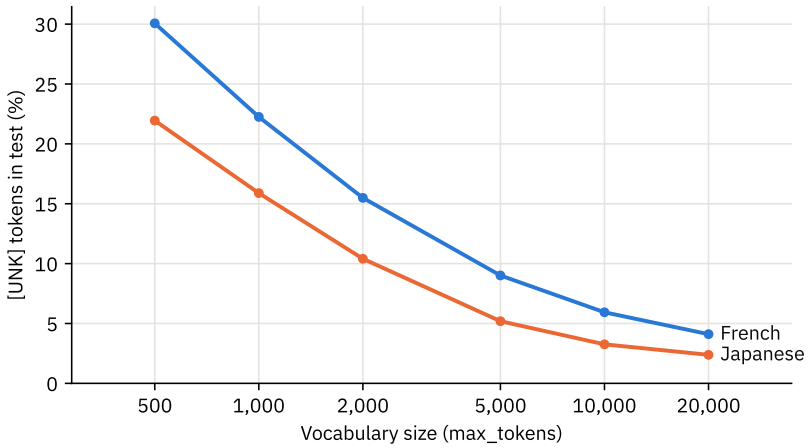


Figure 2: Share of `[UNK]` tokens in the test set, for a given vocabulary size.

`max_tokens`	500	1,000	2,000	5,000	10,000	20,000
French	30.1 %	22.3 %	15.5 %	9.0 %	5.9 %	4.1 %
Japanese	21.9 %	15.9 %	10.4 %	5.2 %	3.3 %	2.4 %

For every vocabulary size, Japanese split by MeCab has fewer unknown tokens than French split on spaces. With 5,000 tokens, 9 % of French tokens are unknown against 5.2 % in Japanese. Once MeCab has done its job, Japanese is not harder to cover than French; on this corpus it is even easier.

3.5 What spaces miss in French

Spaces are not perfect either. When the punctuation is removed, an apostrophe or a hyphen glues two words into one token. On this corpus, that gives **3,258** distinct tokens, and **9 %** of all French tokens. The most common ones:

`jai` `cest` `quil` `nest` `sil` `sest` `ma` `estce` `nai` `na`

j'ai and *ai* become two unrelated tokens. Worse, *m'a* becomes *ma*, the same token as the possessive *ma* (*my*): two different words now share one id. French also needs a little preprocessing, just less than Japanese.

3.6 What MeCab outputs in Japanese

The extra Japanese tokens are mostly grammar. Among the tokens MeCab outputs (punctuation excluded), **33 %** are particles (は, が, を, に…) and **13 %** are auxiliary verbs (です, ます, た…). These are very frequent and few in number, which also explains why the Japanese vocabulary stays smaller: a large part of every sentence is made of tokens the model has already seen many times.

4 Multilingual tokenizers

Current language models do not use spaces or MeCab. They use a *subword* tokenizer, learned once on text from about a hundred languages, and apply it to every language in the same way. Two common ones are tested here on the same Tatoeba pairs (`corpus/multilingual.py`): `bert-base-multilingual-cased` (WordPiece) and `xlm-roberta-base` (SentencePiece).

Mean tokens per sentence	French	Japanese	Japanese longer in
Spaces / MeCab (section 3)	7.6	9.8	—
mBERT	12.2	13.6	61 % of pairs
XLM-R	11.9	10.5	29 % of pairs

On the example pair:

mBERT `Nous` `nous` `somme` `##s` `dé` `##p` `##êché` `##s` `pour` `att` `##rap` `##er` `le` `bus` `.`

`私` `たちは` `##　バス` `##　に` `乗` `る` `##　ために` `急` `いだ` `。`

XLM-R `_Nous` `_nous` `_sommes` `_dép` `ê` `ché` `s` `_pour` `_att` `ra` `per` `_le` `_bus` `.`

`_` `私たちは` `バス` `に` `乗` `るために` `急` `い` `だ` `。`

MeCab `私` `たち` `は` `バス` `に` `乗る` `ため` `に` `急い` `だ` `。`

(## marks a piece that continues the previous one; _ marks the start of a word.)

Neither tokenizer knows anything about French or Japanese grammar. They cut where their training data made it useful: *dépêchés* becomes four pieces, 乗る (*to ride*) is cut in the middle while ために (*in order to*) is kept whole. The two tokenizers do not even agree on which language is longer: with mBERT, Japanese needs more tokens than French in 61 % of the pairs, with XLM-R only in 29 %. mBERT puts every kanji in its own token before splitting the rest; XLM-R learned longer Japanese pieces such as 私たちは (*we* + topic particle). For a model, this matters directly: more tokens means a shorter maximum text length and more computation for the same content.

5 Conclusion

Splitting on spaces works for French and fails completely for Japanese: with `TextVectorization`'s default settings, a Japanese sentence is a single token. Splitting by character avoids unknown tokens but loses the words. A morphological analyzer like MeCab gives back real words, and from there the same pipeline as French can be used.

On 38,292 parallel sentences, Japanese split by MeCab is not harder to handle than French split on spaces. Its sentences are shorter in characters but a little longer in tokens, because particles and auxiliaries become separate tokens. In return, its vocabulary is smaller and it has fewer unknown tokens for any vocabulary size. French has its own

blind spots: elisions and hyphens glue words together (*jai*, *cest*) and can even merge two different words into one token (*ma*).

The difference between the two languages is therefore less about the languages themselves than about the preprocessing each one needs: Japanese needs a dictionary-based analyzer, French gets by with spaces plus a few rules for apostrophes. Multilingual subword tokenizers remove that step by learning the cuts from data, but the result depends heavily on how they were trained: two of them disagree on whether French or Japanese takes more tokens for the same sentence.

These results come from short, everyday sentences. On longer or more technical text (news, Wikipedia), the vocabularies would be larger and MeCab would meet more words missing from its dictionary, so the gap between the two languages could change.